

COMPUTER VISION – ASSESSMENT TOOL 1

EXPERIMENT ANSWERS

Course Code: ITA05

Name:Shalini Douglas

Reg No:192321019

Experiment 1: Feature Descriptor Comparison – SIFT vs Harris

Aim

To compare Harris Corner Detection and SIFT feature extraction on the same image set and evaluate feature quality, robustness, repeatability, and suitability for image matching.

Theory / Principle

Harris detects corners by examining how image intensity changes when a small window is shifted in different directions. A strong response in two orthogonal directions indicates a corner. SIFT (Scale-Invariant Feature Transform) detects stable keypoints across scale space and builds a local gradient-based descriptor around each keypoint. Harris is primarily a corner detector, whereas SIFT provides keypoints together with descriptors that can be matched between images.

Algorithm / Procedure

1. Read the same grayscale image for both methods.
2. Apply Harris corner detection and threshold the corner-response map.
3. Apply SIFT to detect keypoints and compute descriptors.
4. Create a transformed version of the image using rotation and scale change.
5. Match SIFT descriptors using a nearest-neighbor matcher and a ratio test.
6. Compare localization, number of detected features, repeatability, and matching usefulness.

Harris corner response (red markers)

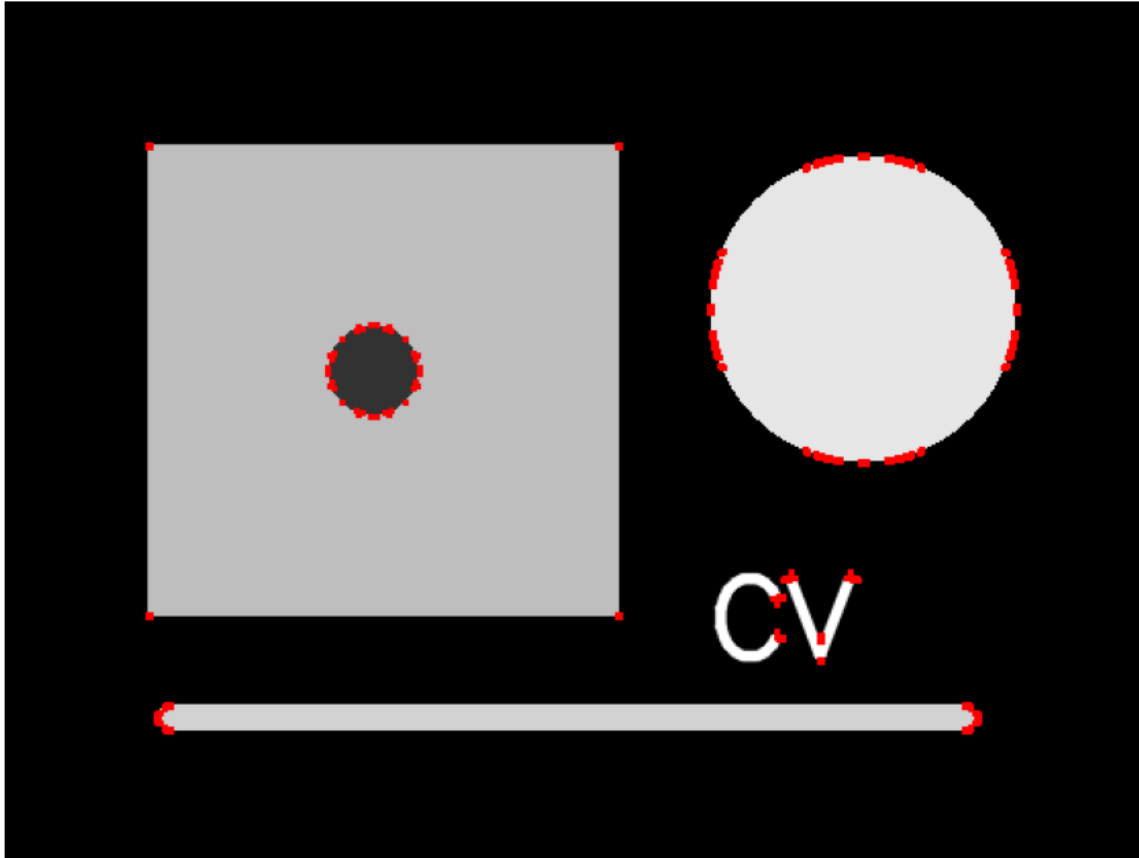


Figure 1.1 – Example Harris corner detections.

SIFT keypoints (n = 45)

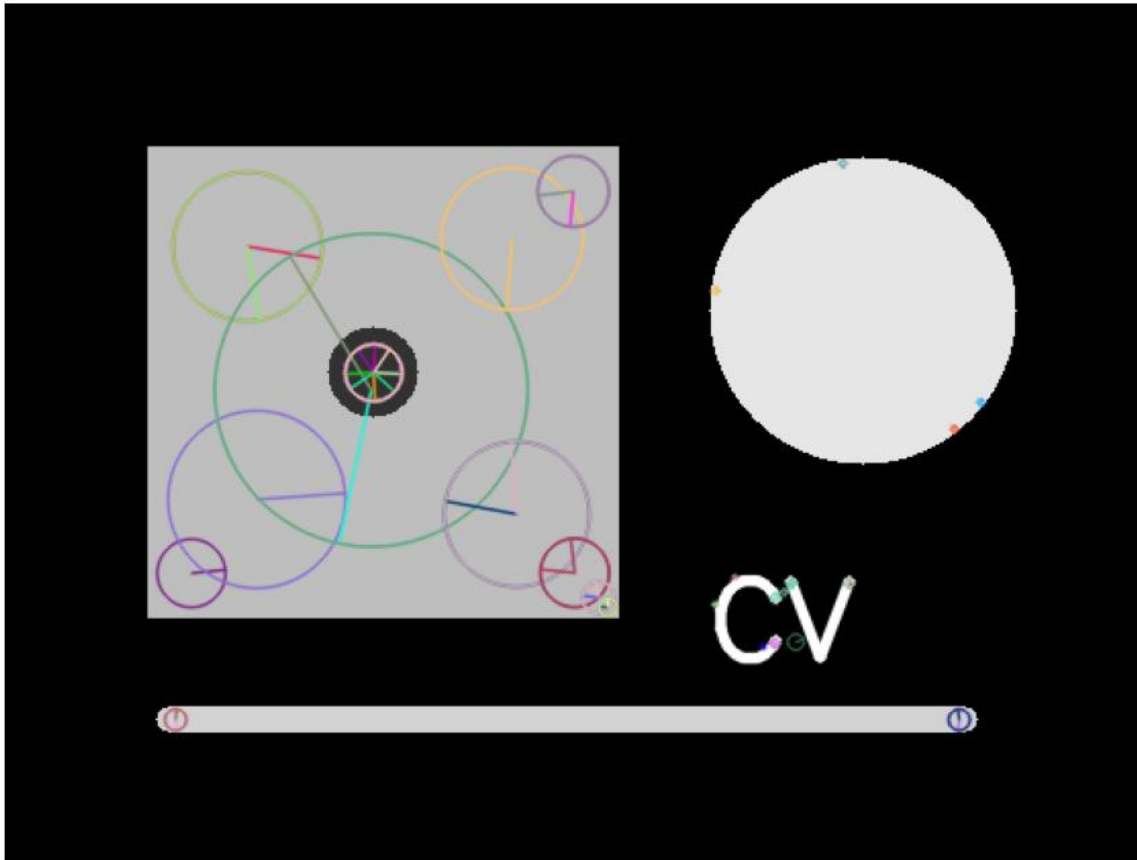


Figure 1.2 – Example SIFT keypoints with scale/orientation visualization.

SIFT matching under rotation/scale (good matches shown: 18)

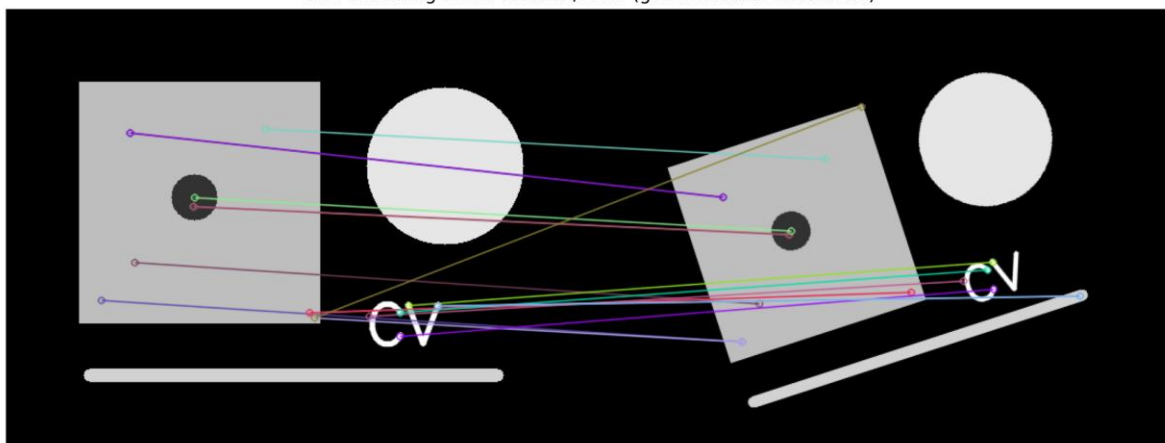


Figure 1.3 – Example SIFT matches after rotation and scaling.

Example OpenCV Implementation

```
import cv2
```

```
img = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)
```

```

# Harris
gray = cv2.float32(img)
h = cv2.cornerHarris(gray, 2, 3, 0.04)

# SIFT
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(img, None)

# Matching
bf = cv2.BFMatcher()
matches = bf.knnMatch(descriptors, descriptors2, k=2)
good = [m for m, n in matches if m.distance < 0.75*n.distance]

```

Comparison and Analysis

Criterion	Harris	SIFT	Observation
Feature type	Corners	Keypoints + descriptors	SIFT contains information suitable for matching.
Scale robustness	Limited	High	SIFT is designed for scale-space detection.
Rotation robustness	Limited / depends on implementation	High	SIFT assigns an orientation to keypoints.
Descriptor	No distinctive descriptor by itself	128-dimensional gradient descriptor	SIFT is directly usable for feature matching.
Computational cost	Lower	Higher	Harris is simpler and generally faster.
Best use	Corner localization	Image matching / recognition	Choice depends on application requirements.

Result: Harris gives accurate corner localization with a relatively simple computation, while SIFT gives more distinctive and robust features for matching across scale and rotation. Therefore, SIFT is generally more suitable when the final task is image matching, while Harris is useful when the main requirement is corner detection.

Experiment 2: Image Enhancement for Feature Detection

Aim

To evaluate how histogram equalization and contrast enhancement influence feature visibility and feature-detection performance.

Theory / Principle

Image enhancement improves the visibility of useful structures before feature extraction. Histogram equalization redistributes intensity values to increase global contrast. Local contrast enhancement such as CLAHE operates on small regions and limits excessive amplification of noise. Better-separated edges and textures can make keypoint detection more reliable, although aggressive enhancement may also strengthen unwanted noise.

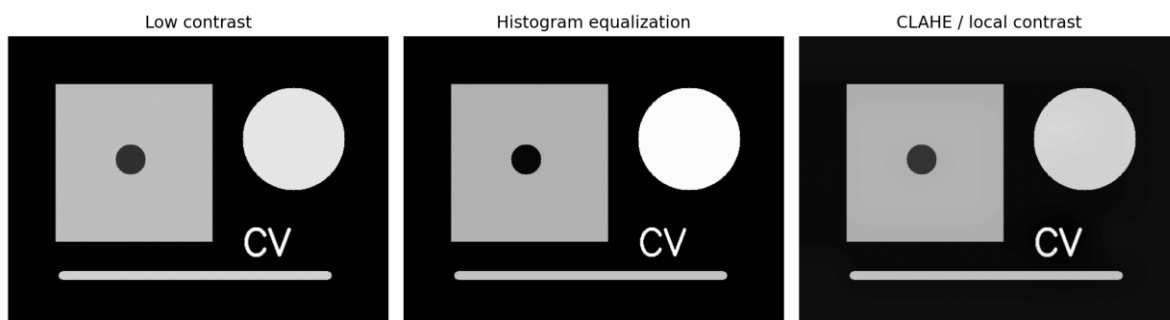


Figure 2.1 – Example enhancement comparison.

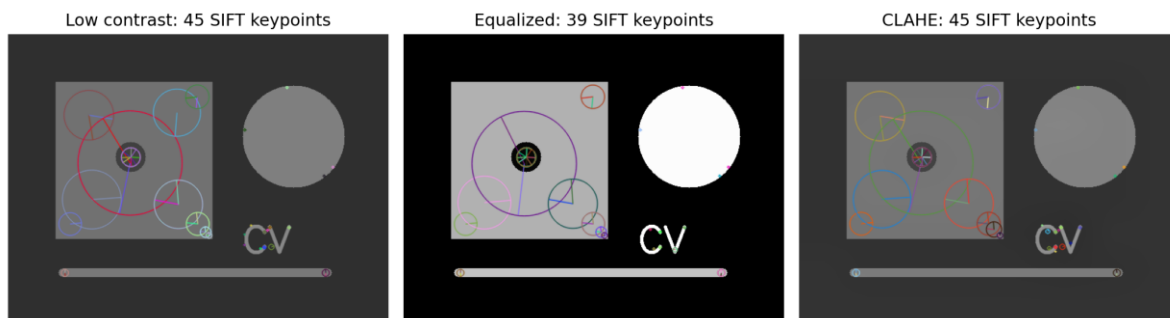


Figure 2.2 – Example SIFT detections after different enhancement methods.

Procedure

7. Read an input image and convert it to grayscale.
8. Create a low-contrast version or use an image with uneven illumination.
9. Apply global histogram equalization.
10. Apply a local contrast method such as CLAHE.
11. Run the same feature detector on the original and enhanced images.
12. Compare feature visibility, number of keypoints, localization, and stability.

Example OpenCV Implementation

```
gray = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)
```

```
equalized = cv2.equalizeHist(gray)
```

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
```

```
local_enhanced = clahe.apply(gray)
```

```
sift = cv2.SIFT_create()
```

```
kp1, des1 = sift.detectAndCompute(gray, None)
```

```
kp2, des2 = sift.detectAndCompute(equalized, None)
```

```
kp3, des3 = sift.detectAndCompute(local_enhanced, None)
```

Observation

Enhancement can expose weak edges and texture that are difficult to distinguish in a low-contrast image. Global equalization is simple and effective for broad contrast changes, whereas local enhancement is useful when illumination varies spatially. The best method should be selected using the stability and usefulness of the detected features rather than simply the visual brightness of the image.

Method	Advantage	Possible limitation
Histogram equalization	Improves global contrast	May over-enhance noise or bright regions
Contrast adjustment	Simple control of intensity range	Can clip dark/bright details
CLAHE	Improves local contrast	Extra computation; can amplify noise if too strong

Result: Image enhancement can improve feature visibility and detection reliability, but parameters must be chosen carefully to avoid introducing unstable features.

Experiment 3: Multi-Scale Feature Detection Analysis

Aim

To study feature detection at different image scales and analyze feature stability, accuracy, and computational behavior.

Theory / Principle

Image scaling changes spatial resolution. A scale-space representation analyzes an image over multiple resolutions, allowing features that appear at different physical sizes to be detected more consistently. Downsampling reduces the number of pixels and can reduce computation, but very small structures may disappear. A robust multi-scale detector should preserve important structures while avoiding unnecessary repeated computation.

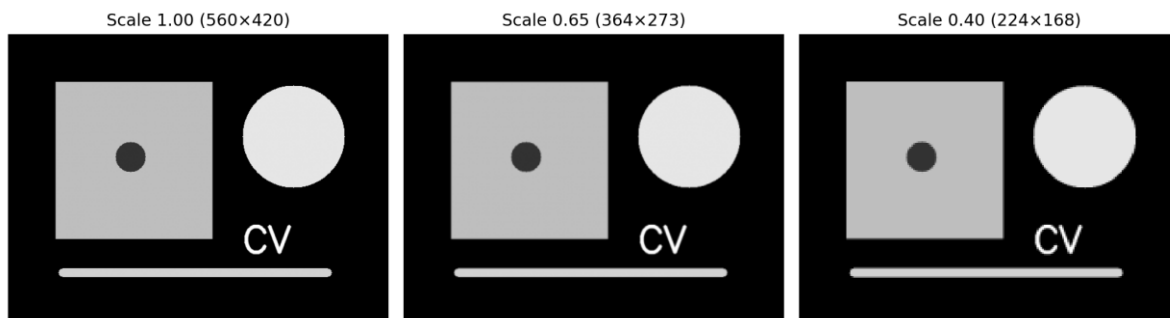


Figure 3.1 – Example image at three scales.

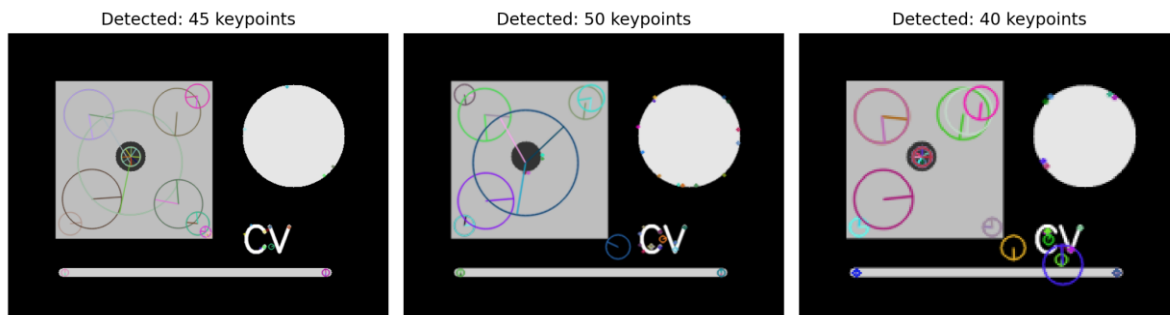


Figure 3.2 – Example SIFT keypoints detected at different scales.

Procedure

13. Start with a reference image.
14. Generate reduced-resolution versions such as 100%, 65%, and 40%.
15. Run the same detector on each scale.
16. Record image dimensions, number of keypoints, processing time, and whether important structures remain detectable.
17. Compare stability of keypoints across scales.

Example Implementation

```
scales = [1.0, 0.65, 0.40]
```

```
sift = cv2.SIFT_create()
```

```
for s in scales:
```

```
    small = cv2.resize(img, None, fx=s, fy=s,
```

```
                        interpolation=cv2.INTER_AREA)
```

```
    keypoints, descriptors = sift.detectAndCompute(small, None)
```

```
    print(s, small.shape, len(keypoints))
```

Analysis

At lower resolutions, large structures generally remain detectable while fine details may disappear. The number of keypoints can therefore change with scale. Smaller images usually require less computation, but excessive downsampling can reduce localization accuracy and remove useful features. SIFT addresses scale variation by detecting features in a scale-space representation rather than relying on only one image size.

Result: Multi-scale analysis improves understanding of feature stability and helps select an image resolution that balances detection quality and computational cost.

Experiment 4: Comparative Analysis of Image Filtering Techniques

Aim

To compare Mean, Gaussian, and Median filtering for preprocessing, with emphasis on noise reduction and preservation of important image features.

Theory / Principle

Mean filtering replaces each pixel with the average of a neighborhood and therefore smooths random noise, but it can blur edges. Gaussian filtering performs weighted smoothing, giving larger weights to nearby pixels and usually producing a natural blur. Median filtering replaces each pixel with the median of its neighborhood and is particularly effective for impulse (salt-and-pepper) noise while preserving edges better than simple averaging.

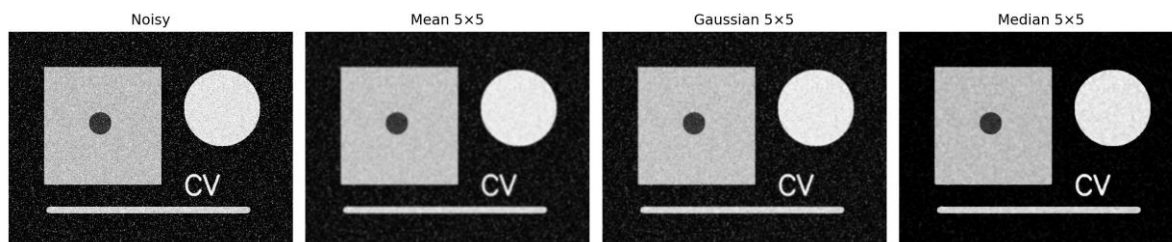


Figure 4.1 – Example output of Mean, Gaussian, and Median filtering.

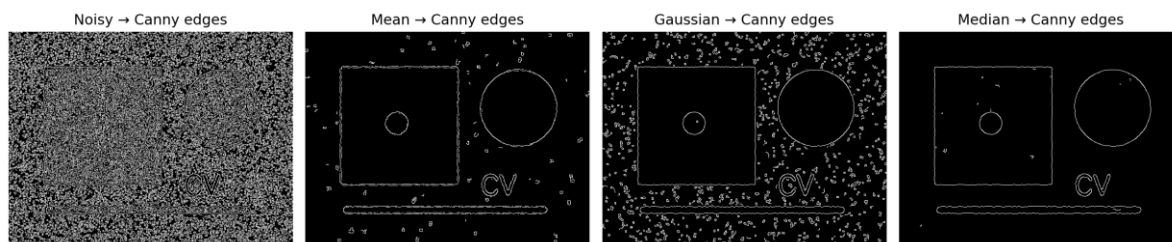


Figure 4.2 – Example Canny edges after each filtering method.

Procedure

18. Add or obtain a noisy grayscale image.
19. Apply a 5×5 Mean filter.
20. Apply a 5×5 Gaussian filter.
21. Apply a 5×5 Median filter.
22. Compare smoothness, residual noise, and edge preservation.
23. Apply an edge detector after filtering and inspect the resulting edges.

Example Implementation

```
mean_img = cv2.blur(noisy, (5, 5))
gaussian_img = cv2.GaussianBlur(noisy, (5, 5), 1.2)
median_img = cv2.medianBlur(noisy, 5)
```

```
edge_mean = cv2.Canny(mean_img, 80, 160)
edge_gauss = cv2.Canny(gaussian_img, 80, 160)
edge_median = cv2.Canny(median_img, 80, 160)
```

Filter	Main operation	Noise behavior	Edge behavior
Mean	Neighborhood average	Good for mild random noise	More blur at edges
Gaussian	Weighted neighborhood average	Good for Gaussian-like noise	Smoother, controlled blur
Median	Neighborhood median	Very effective for salt-and-pepper noise	Usually preserves edges better

Result: Median filtering is often the strongest choice for impulse noise, Gaussian filtering is a useful general smoothing operation, and Mean filtering is simple but can soften important boundaries. The correct choice depends on the noise model and the feature-detection stage that follows.

Experiment 5: Complete Object Detection Workflow

Aim

To develop an integrated object-detection workflow containing image preprocessing, edge detection, feature extraction, and shape detection.

Theory / Principle

A complete detection pipeline combines several stages. Preprocessing reduces unwanted variation, edge detection highlights object boundaries, feature extraction provides informative local structures, and shape detection converts image evidence into object hypotheses. The stages interact: good preprocessing improves edges, good edges support reliable shape analysis, and stable features help distinguish objects under changing image conditions.

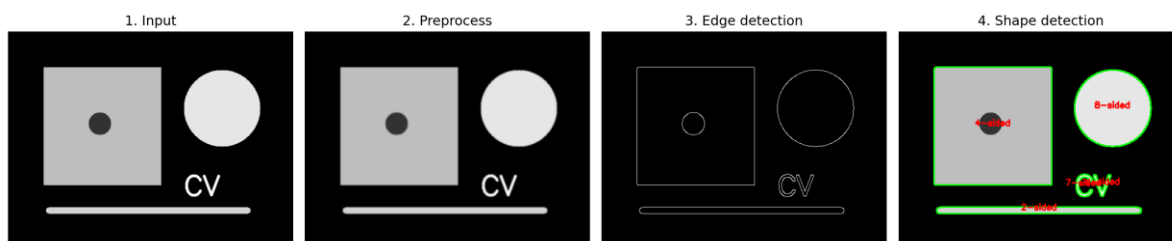


Figure 5.1 – Example integrated workflow from input to shape detection.

Pipeline

24. Input image acquisition.
25. Convert to grayscale and optionally resize.
26. Reduce noise using an appropriate filter.
27. Detect edges using Canny or another edge detector.
28. Extract local features/keypoints if required.
29. Detect shapes using contours, Hough lines/circles, or another shape model.
30. Validate detections using geometry, size, location, or descriptor matching.
31. Report final object detections.

Example OpenCV Implementation

```
img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
smooth = cv2.GaussianBlur(gray, (5, 5), 1.2)
edges = cv2.Canny(smooth, 80, 160)
contours, _ = cv2.findContours(
    edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for c in contours:
    if cv2.contourArea(c) > 250:
        peri = cv2.arcLength(c, True)
        approx = cv2.approxPolyDP(c, 0.04 * peri, True)
```

Stage-wise Analysis

Stage	Purpose	Effect on detection
Preprocessing	Reduce noise / normalize appearance	Produces cleaner boundaries
Edge detection	Find intensity transitions	Highlights object outlines
Feature extraction	Represent informative local structures	Supports recognition and matching
Shape detection	Infer geometric objects	Converts low-level evidence into object candidates
Validation	Reject false positives	Improves precision and reliability

Overall Result

The integrated workflow is more useful than applying a single operation because each stage prepares information for the next stage. Under different image conditions, preprocessing parameters, edge thresholds, feature settings, and shape constraints should be adjusted and validated using detection quality. The uploaded assessment specifically asks that outputs be documented at every stage and that the integrated approach be analyzed for accurate and reliable detection.